

Generative AI for Multimodal Visual Intelligence

Project Workbook 1

CMPE 295A: Master Project I

Department of Computer Engineering

Charles W. Davidson College of Engineering

San José State University

Prepared by:

Sophia Atendido (016145753)

Raymond Li (015611024)

Jack Liang (016885414)

Jeremy Tung (015591823)

Project Advisor:

Professor Vidhyacharan Bhaskar

Date:

October 1, 2026

Table of Contents

Chapter 1: Literature Search, State of the Art (Jeremy Tung).....	2
1.1 Literature Search.....	2
1.2 State-of-the-Art Summary.....	3
1.3 References.....	4
Chapter 2: Project Justification (Jeremy Tung).....	6
Chapter 3: Project Requirements (Raymond Li).....	8
3.1 Use Cases & Traceability.....	8
3.2 Functional Requirements.....	8
3.3 Non-Functional Requirements.....	10
Chapter 4: Dependencies and Deliverables (Raymond Li).....	12
4.1 Dependencies.....	12
4.2 Deliverables.....	12
Chapter 5: Project Architecture (Jack Liang).....	14
Chapter 8: Implementation Plan and Progress (Sophia Atendido).....	16
Chapter 9: Project Schedule (Sophia Atendido).....	19

Chapter 1: Literature Search, State of the Art (Jeremy Tung)

1.1 Literature Search

Transformer and graph-based models remain standard approaches for building visual models. For instance, Jang et al. developed a sign-language translation system that pairs visual features from continuous signing with contextual data and a pretrained large language model [1]. This demonstrates how generative models can leverage diverse visual contexts to significantly improve their outputs. On the other hand, visual models have also evolved to handle complex networks; Moon and Cho introduced a graph Transformer for traffic prediction that captures both the spatial layout of sensors and shifting traffic conditions over time [2]. Together, these studies highlight the versatility of modern visual sequence models in domains as different as sign language interpretation and traffic analysis.

More recently, Mamba has emerged as a compelling alternative to Transformers. Introduced by Gu and Dao, it is a selective state-space model that handles long sequences with linear computational complexity [3]. Li et al. adapted this for video with VideoMamba, using state-space modeling to capture both quick, short-term motions and long-range temporal patterns [4]. Crucially, this avoids the quadratic attention costs of standard video Transformers—making it especially promising for sign language recognition and traffic analysis, where context unfolds across long stretches of consecutive frames.

Meanwhile, the shift toward public datasets has been a major catalyst for isolated sign recognition. Li et al. first pushed the field forward with WLASL, a video dataset of over 2,000 ASL signs; they found that while pose- and appearance-based models perform well initially, their accuracy drops off significantly as the vocabulary expands [5]. Later, Desai et al. released ASL Citizen, the first crowdsourced dataset of its kind, featuring over 80,000 videos across 2,731 signs recorded in everyday environments [6]. Google also advanced the space by hosting an Isolated Sign Language Recognition competition on Kaggle, releasing nearly 95,000 clips using MediaPipe landmarks rather than raw pixels [7]. Google's MediaPipe framework is highly efficient at extracting hand, face, and body landmarks on-device in real time [8]. And as Bohacek and Hruz demonstrated with SPOTER—a Transformer built purely for body pose sequences—operating on landmarks rather than pixels can deliver competitive accuracy at a fraction of the compute [9]. These findings directly inform our project: we train on ASL Citizen landmarks using a signer-independent split, and classify signs using the landmark-based Transformer architecture originally introduced by Vaswani et al. [10].

In the traffic domain, standard vehicle perception pipelines typically kick off with a fast, one-stage detector like YOLO [11]. This is usually paired with a tracker like ByteTrack to keep vehicle identities consistent from frame to frame [12]. For traffic forecasting, Li et al. set a widely used baseline by modeling traffic flow as a diffusion process across a road network using DCRNN [13]. When it comes to actually controlling signals, reinforcement learning methods like IntelliLight have been used to derive policies directly from real-world data [14]. Before these strategies ever see a real intersection, they are rigorously tested in SUMO, an open-source

microscopic traffic simulator built by the German Aerospace Center [15]. Our project adopts this familiar detect-track-forecast-simulate pipeline. However, instead of relying on a learned control policy, we use a language model to generate signal plans that are independently verified by code.

1.2 State-of-the-Art Summary

Today's state-of-the-art AI systems are inherently multimodal, smoothly processing combinations of text, images, and video. Vision-language models, in particular, have become a standard bridge for translating physical movement into text or speech. Crucially, these systems are no longer restricted to still images or brief clips. For instance, OpenAI recently demonstrated that GPT-4.1 achieves strong performance on Video-MME, a benchmark designed to evaluate comprehension of videos up to an hour long [16]. This capacity to ingest and parse extended content opens up entirely new possibilities for continuous video analysis.

At the same time, new architectures are emerging to make processing these long sequences dramatically more efficient. Traditional video Transformers rely on self-attention mechanisms that suffer from a quadratic increase in compute cost as a video gets longer. To solve this, models like VideoMamba leverage state-space modeling to process temporal data with linear complexity. This allows them to capture both immediate, fine-grained motions and sweeping, long-range dependencies across extended video sequences. Because both sign language interpretation and traffic analysis fundamentally rely on tracking how visual context evolves—rather than just analyzing static frames—these efficiency gains are incredibly valuable.

Beyond perception, generative AI is increasingly stepping into active decision-making roles. In intelligent transportation, recent approaches have started framing large language models as reasoning agents capable of directing traffic. A study in IEEE Transactions on Intelligent Transportation Systems demonstrated this by using LLM-based agents for adaptive signal control; the system dynamically adjusted signal patterns in response to real-time conditions, noticeably reducing stops and boosting average vehicle speeds compared to traditional methods [17].

The industry is already shifting to reflect these capabilities. Google's Project Green Light analyzes driving trends from Google Maps to suggest optimal signal timings to city engineers. Running in major cities like Seattle, Boston, Rio de Janeiro, and Bengaluru, the system has reportedly reduced intersection stops by up to 30% and emissions by 10% [18]. This human-in-the-loop model—where the AI recommends a plan but an engineer holds the final say—mirrors the approach we use for our own signal agent. Similar leaps are happening in accessibility; in 2025, Google DeepMind introduced SignGemma, an open model specifically trained to translate sign language into spoken text [19]. Finally, the rise of parameter-efficient fine-tuning techniques like QLoRA means these powerful open models can now be adapted for highly specific tasks—like parsing food orders—using just a single consumer GPU [20].

1.3 References

- [1] Jang, Y., Raajesh, H., Momeni, L., Varol, G., & Zisserman, A. (2025). Lost in translation, found in context: Sign language translation with contextual cues. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 8742–8752).
- [2] Moon, H.-J., & Cho, S.-B. (2025). Traffic prediction by graph transformer embedded with subgraphs. *Expert Systems with Applications*, 272, 126799. <https://doi.org/10.1016/j.eswa.2025.126799>
- [3] Gu, A., & Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces. In *Proceedings of the Conference on Language Modeling*.
- [4] Li, K., Li, X., Wang, Y., He, Y., Wang, Y., Wang, L., & Qiao, Y. (2024). VideoMamba: State space model for efficient video understanding. In *Computer Vision – ECCV 2024* (pp. 237–255). Springer. https://doi.org/10.1007/978-3-031-73347-5_14
- [5] Li, D., Rodriguez, C., Yu, X., & Li, H. (2020). Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (pp. 1459–1469).
- [6] Desai, A., Berger, L., Minakov, F., Milano, N., Singh, C., Pumphrey, K., Ladner, R. E., Daumé III, H., Lu, A. X., Caselli, N., & Bragg, D. (2023). ASL Citizen: A community-sourced dataset for advancing isolated sign language recognition. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*.
- [7] Google. (2023). Google - Isolated Sign Language Recognition [Kaggle competition]. <https://www.kaggle.com/competitions/asl-signs>
- [8] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172.
- [9] Bohacek, M., & Hruz, M. (2022). Sign pose-based Transformer for word-level sign language recognition. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops* (pp. 182–191).
- [10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998–6008).
- [11] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779–788).
- [12] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *Computer Vision - ECCV 2022* (pp. 1–21). Springer.

- [13] Li, Y., Yu, R., Shahabi, C., & Liu, Y. (2018). Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In Proceedings of the International Conference on Learning Representations.
- [14] Wei, H., Zheng, G., Yao, H., & Li, Z. (2018). IntelliLight: A reinforcement learning approach for intelligent traffic light control. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 2496–2505).
- [15] Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flotterod, Y.-P., Hilbrich, R., Lucken, L., Rummel, J., Wagner, P., & Wiessner, E. (2018). Microscopic traffic simulation using SUMO. In Proceedings of the 21st International Conference on Intelligent Transportation Systems (ITSC) (pp. 2575–2582). IEEE.
- [16] OpenAI. (2025). Introducing GPT-4.1 in the API. Retrieved from <https://openai.com/index/gpt-4-1/>
- [17] Wang, X., et al. (2025). The crossroads of LLM and traffic control: A study on large language models in adaptive traffic signal control. IEEE Transactions on Intelligent Transportation Systems, 26(2), 1701–1716. <https://doi.org/10.1109/TITS.2024.3498735>
- [18] Marini, A. (2024, July 29). How Project Green Light uses AI to reduce gas emissions. Google Blog. <https://blog.google/company-news/outreach-and-initiatives/sustainability/google-ai-project-green-light/>
- [19] Google DeepMind. (2025, May 27). Announcing SignGemma, a model for translating sign language into spoken text [Post]. X. <https://x.com/GoogleDeepMind/status/1927375853551235160>
- [20] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In Advances in Neural Information Processing Systems 36.
- [21] United Nations. (n.d.). International Day of Sign Languages, 23 September. <https://www.un.org/en/observances/sign-languages-day>
- [22] INRIX. (2025). INRIX 2024 Global Traffic Scorecard: Employees and consumers returned to downtowns, traffic delays and costs grew. <https://inrix.com/press-releases/2024-global-traffic-scorecard-us/>

Chapter 2: Project Justification (Jeremy Tung)

The purpose of this project is to explore how one generative AI framework can be applied to two visually different but technically related problems: sign-language interpretation and traffic analysis. Although these applications appear unrelated, both require a system to process continuous visual information, understand how that information changes over time, and generate a useful output based on that context. In the sign-language application, the output is translated language, while in the traffic application, the output is a recommendation or control decision.

The project focuses on Gen AI because the goal is not only to classify what appears in a video, but to interpret the visual sequence and produce a new, context-aware output. Modern systems are often designed for a single application, such as sign recognition, object detection, or traffic prediction. This project instead investigates whether a shared architecture using visual perception, temporal modeling, and generative reasoning can be adapted across multiple domains.

The project also tests whether one shared temporal model can serve both tasks. Each task gets a small input adapter and output head, while the core model stays the same. The question is whether that shared model can still keep the accuracy each task needs. The overall goal is therefore not to replace existing specialized systems, but to demonstrate a more unified approach to real-time visual understanding and generative decision-making.

Both problems carry real and measurable costs. The United Nations estimates that more than 70 million deaf people worldwide use over 300 different sign languages [21], yet self-service ordering kiosks, which are now common in quick-service restaurants, are designed around touch and, more and more, voice. That leaves signers without a natural way to order. On the traffic side, INRIX estimates that the average U.S. driver lost 43 hours to congestion in 2024, costing the country \$74 billion in lost time [22]. A system that helps in either domain has clear practical value, and one design that works in both is a stronger result than two separate tools.

Concretely, both applications follow the same three-stage pipeline. A perception stage reduces raw video to a compact structured representation: hand and body landmarks for signing, and vehicle positions and counts for traffic. A temporal stage models how that representation changes over a window of frames, because in both domains the meaning lives in the change rather than in any single frame. A generative stage then turns the temporal model's output into an action expressed in natural language, and a non-generative verifier constrains what that action can be.

That verifier is the strongest shared element and the main contribution of the project. In the ordering terminal, the language model may propose items, but prices are always read from the database, so a price can never be invented. This is already enforced by an automated price-integrity test in the codebase. In the traffic module, the language model may propose a signal timing, but code checks it against minimum green and clearance times before it is shown. In both cases, the generative component controls the wording and has no control over

correctness. Testing this one design across two very different domains is what makes the project a research question rather than two separate applications.

The project is also set up to produce publishable results. These include signer-independent accuracy and per-signer fairness results on a public benchmark, results for one shared model trained on both ASL and traffic data, a benchmark of cart accuracy, price integrity, and latency for LLM ordering agents, and a simulation study of a verified LLM signal agent against fixed timing. Each of these fits established venues, such as ACM ASSETS for accessibility, the IEEE Intelligent Transportation Systems Conference for traffic, and the sign language and human-centered vision workshops at CVPR and WACV. Because camera images never leave the device and the language model can only recommend, the design also fits funding programs focused on accessible, privacy-preserving, human-in-the-loop AI.

Chapter 3: Project Requirements (Raymond Li)

3.1 Use Cases & Traceability

ID	Use case	Actor
UC1	Walk up to the terminal, be noticed, and be greeted by voice, on-screen, and with a signed greeting.	Customer
UC2	Place a full order by signing in ASL.	ASL user
UC3	Place a full order by speaking.	Customer
UC4	Place a full order by touch.	Customer
UC5	Review the cart, change or remove an item, and confirm the order.	Customer
UC6	Accept or decline a suggested special or add-on.	Customer
UC7	Play a recorded intersection video and see vehicle counts and queue lengths.	Traffic operator
UC8	Get a recommended signal timing with a short plain-language reason.	Traffic operator
UC9	Run the simulator to compare the recommended timing with fixed timing.	Team
UC10	Run the same trained model on an ASL clip and on an intersection clip.	Team

3.2 Functional Requirements

ID	Requirement	Priority
F1	The terminal takes a complete order by ASL, speech, or touch. Each channel works on its own.	Essential
F2	The terminal detects a person approaching and greets them in every mode at once, then adapts to the mode they use.	Essential
F3	The browser runs MediaPipe locally and sends only hand and upper-body pose landmarks to the server. No camera images leave the device.	Essential
F4	The ML service splits the landmark stream into signs and classifies them. The current model covers 64 signs that cover menu items, quantities, and ordering actions, and the vocabulary is extensible as more classes are trained.	Essential

ID	Requirement	Priority
F5	The agent turns signs, speech, and touch into structured cart actions, including ASL gloss order such as WANT / BURGER / TWO.	Essential
F6	Menu items and prices come only from the database through tool calls. The language model never generates an item or a price.	Essential
F7	The customer can review, change, and confirm the cart, and the confirmed order is saved.	Essential
F8	Every step is shown on screen and spoken aloud.	Essential
F9	The shared model's sign head is trained with a signer-independent split, and accuracy is reported per sign and per signer.	Essential
F10	The traffic module detects and tracks vehicles in recorded intersection video and builds a traffic state (vehicle count and queue length per approach).	Essential
F11	One shared model is trained on both ASL landmark sequences and traffic sequences. Each task has a small input adapter and output head (signs for ASL, short-term forecast for traffic).	Essential
F12	The shared model is tested on held-out data for both tasks: ASL accuracy per sign and signer, and traffic forecast error on video it was not trained on.	Essential
F13	An LLM signal agent proposes a signal timing from the traffic state and forecast and explains it. Code checks every plan against minimum green and clearance times before it is shown.	Essential
F14	The signal agent is tested in a traffic simulator against a fixed-timing baseline, using stopped vehicles and average speed.	Essential
F15	An order recommender suggests specials and add-ons from the current cart.	Desired
F16	A model fine-tuned with QLoRA on synthetic ordering dialogues runs locally through Ollama and is benchmarked against a hosted model.	Desired
F17	When a sign is uncertain, the terminal asks the customer to confirm before adding it.	Desired
F18	A recording tool captures the missing ordering signs for training.	Desired
F19	An ordering benchmark scores cart match, price integrity, and latency for each agent backend.	Desired
F20	A dashboard shows the video, counts, forecast, and recommended signal plan.	Desired

ID	Requirement	Priority
F21	A feedback session with ASL users or Deaf community members reviews the terminal.	Optional
F22	A food-image recognition model as an additional input.	Optional
F23	A physical kiosk setup (touchscreen and camera stand).	Optional
F24	A live camera feed or several linked intersections for the traffic module.	Optional

3.3 Non-Functional Requirements

ID	Requirement	Category
N1	Sign recognition round trip stays under 100 ms on the demo laptop, so the terminal feels conversational.	Performance
N2	No camera imagery is transmitted or stored for signing. Only landmark coordinates leave the device.	Privacy
N3	The benchmark shows zero invented items and zero wrong prices.	Correctness
N4	Target of at least 85% top-1 accuracy on held-out signers for the ordering vocabulary. Current best is 88.6% on 64 classes, from an ASL-only run.	Accuracy
N5	The interface meets WCAG 2.1 AA and works with a keyboard and a screen reader.	Accessibility
N6	If no trained sign model is loaded, sign input stays inactive instead of producing fake signs.	Reliability
N7	Training is repeatable: fixed seeds, saved settings, and documented steps.	Maintainability
N8	Datasets and traffic video are used within their terms. Menu prices are labeled as estimates until replaced.	Legal / honesty
N9	Runs locally in Chrome with Node and Python, without Docker.	Portability
N10	The terminal does not claim to replace a human interpreter and asks for confirmation when unsure.	Safety
N11	The signal agent only recommends. It is never connected to real traffic signals, and every plan stays inside safe timing limits.	Safety
N12	In simulation, the recommended timing reduces average stopped vehicles by at least 10 percent compared with fixed timing, without lowering average speed, across all test scenarios.	Effectiveness

ID	Requirement	Category
N13	Vehicle counts from the detector are within 10 percent of hand counts on at least five sample clips.	Accuracy
N14	After joint training, the shared model still meets N4 and its queue-length forecast has a lower mean absolute error than a historical-average baseline on held-out video.	Accuracy
N15	Both applications load the same saved model file, so there is one model to store and update.	Maintainability

Chapter 4: Dependencies and Deliverables (Raymond Li)

4.1 Dependencies

- **ASL training data.** We plan to train on public ASL datasets like ASL Citizen and Google ISLR. They cover most of the signs we need. If we lose access or the license changes, we can't train the model. We will probably have to record a few signs ourselves.
- **Traffic data.** The traffic part needs video or sensor data we are allowed to use. If we can't find any, we can use a public vehicle dataset or traffic generated by a simulator. The results might not work for other kinds of intersections.
- **One model for two tasks.** Signing and traffic have very different inputs, so one shared model might do worse on one of them. If that happens, we can keep the shared part fixed and train a separate output for each task.
- **Traffic simulator.** We want to test the signal agent in a simulator like SUMO. If we can't set up a realistic intersection, we won't have a fair way to compare it to fixed timing. A simple simulation we build ourselves could be a backup.
- **GPU access.** The ASL model is small, but the shared model, the LLM fine-tune, and the vehicle detector will probably need a GPU. We can use free options like Colab or Kaggle, or ask about SJSU resources.
- **Browser support.** The terminal depends on Chrome for MediaPipe, the camera, and speech. If Chrome changes something, parts of it could break.
- **Processing time.** Getting landmarks from the full ASL dataset is slow. If it takes too long, we will train on a smaller set of signs and signers.
- **Local language model.** The fine-tuned model has to run on our own computers. If it can't, we can use the hosted model instead.
- **Hosted LLM access.** The hosted model needs an API key and some budget. If we don't have that, we can use the rules-based option or the local model, which works offline.
- **Fair testing.** Our accuracy numbers only count if the model never saw the test signers. If anything fails, we would retrain from our saved split and keep track of which version gave each result.

4.2 Deliverables

- **Shared model:** one model trained on both ASL and traffic data, with a small input part and output part for each task.
- **SignOrder terminal:** a web app where you can order by signing, speaking, or touch.
- **ML service** that finds signs in the camera data and recognizes them.
- **Training and evaluation code**, plus our results for ASL and traffic.
- **Order recommender** that suggests extras based on the cart.
- **Fine-tuned ordering language model**, compared against a hosted one.

- **Traffic module:** vehicle detection, a short-term traffic forecast, an LLM that suggests signal timing, and a dashboard.
- **Simulation study** comparing our suggested signal timing to fixed timing.
- **Sign recording tool** for signs we need to record ourselves.
- **Documentation** and the ordering benchmark.
- **Evaluation report** with accuracy results (including across different signers), results for the shared model on both tasks, and the ordering benchmark.
- **Live demo:** ordering in ASL at a laptop webcam, plus the traffic dashboard on recorded video.

Chapter 5: Project Architecture (Jack Liang)

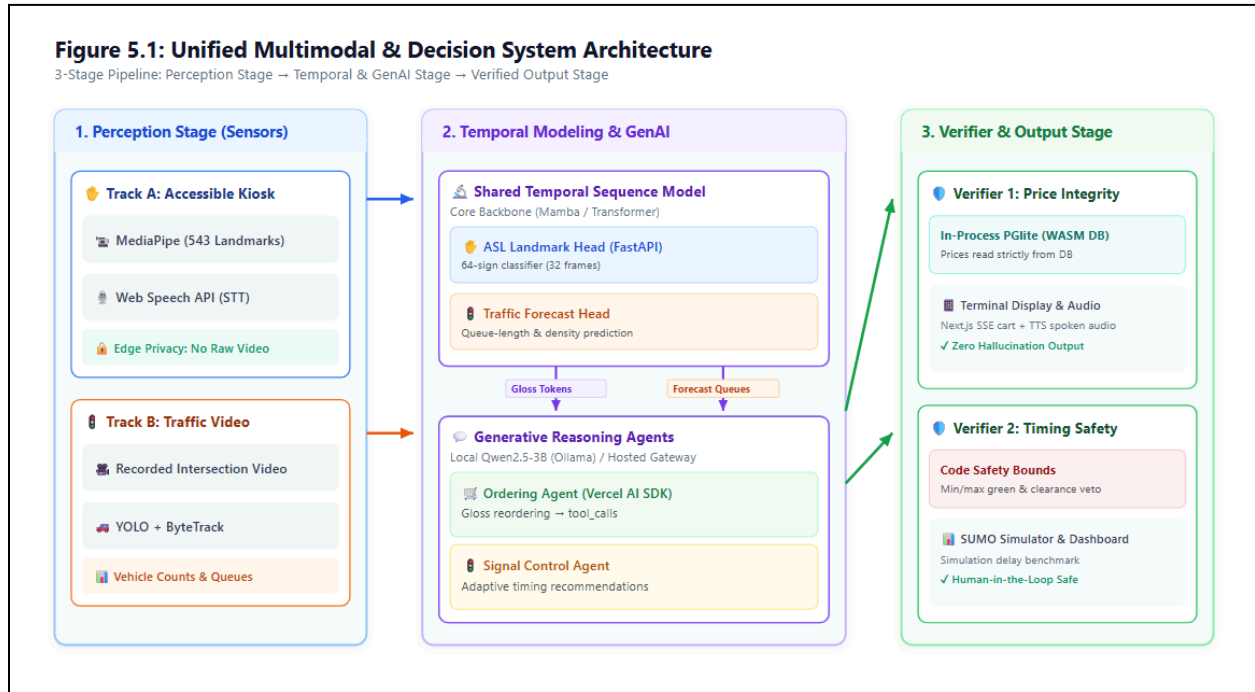


Figure 5.1: System Architecture Overview

Figure 5.1 illustrates the pipeline for our system architecture. Our system is designed around a shared pipeline that handles inputs for both an accessible American Sign Language ordering kiosk and traffic video analysis. The client applications support vision and speech capabilities. From there, data is sent to the server to process using machine learning algorithms and calculate the proper output flow.

The Client Terminal is crucial to maximize user privacy. Because the kiosk application runs directly in the browser, it minimizes server calls for actions that can be handled locally. Instead of streaming raw video over the network constantly, it extracts relevant landmark data points from the image so that the server only has to handle the heavier machine learning processing on numerical data inputs. For traffic, recorded video is processed in Python through YOLO and ByteTrack to extract vehicle tracking data and counts.

The Next.js gateway handles the bridge between the client and server, as it provides the WebSocket connection and manages user sessions and cross-communication. This layer also contains the PGLite database, which is a compilation of PostgreSQL to WebAssembly. This allows the kiosk application to handle database interactions locally without needing a separate database daemon.

On the backend side, the machine learning inference tier handles the computation for the shared temporal sequence model, running heads for both sign recognition and traffic forecasting. The sign head outputs recognized gloss tokens to our ordering agent, while the traffic head outputs

queue forecasts to our signal control agent. For generative reasoning, our agent connects to the language model through either local Ollama or a hosted model to handle order actions and signal timing recommendations. Both outputs are checked by verifiers, ensuring prices come strictly from the database and signal timings stay within safety limits.

For the tech stack, we chose Next.js and React alongside Tailwind CSS for the client. For our core artificial intelligence tools, we chose the Vercel AI SDK to manage agents and the tool-calling loop. Our backend runs on Python to handle machine learning inference and traffic processing, while the Next.js server uses PGlite for database handling.

Our application is intentionally designed to follow a tiered, client-server architecture. Anything related to presentation and tasks able to be handled by the application is decoupled from the machine learning pipeline and the database layer. This separation of concerns helps us run the compute-heavy artificial intelligence models without blocking any of the user interface.

Chapter 8: Implementation Plan and Progress (Sophia Atendido)

8.1 Environment Setup

Area	What is in place	Status
1. Programming and execution environment	• Shared GitHub repo • Web terminal in Next.js, ML service in Python and FastAPI • Run guide in docs/development.md	Done
2. Development tools	• MediaPipe, Web Speech API, and ONNX Runtime in use • Ollama for a local LLM, Kaggle for datasets • SUMO, YOLO, ByteTrack, and PEFT are free and open source, and each member has the tools for their part (see 8.3)	Acquired
3. Hardware and prototyping environment	• Laptop with a webcam and Chrome for demos • Faster computer for landmark extraction • Free Colab or Kaggle GPU for training	In place
4. Example programs	• MediaPipe run on ASL videos to get landmarks • Tests for the segmenter and a health check for the ML service	Done
5. Prototype	• Walking skeleton: browser to server to agent to spoken reply • Sign input stays off if no model is loaded • First small test of each remaining tool is planned in 8.5	In progress

8.2 Component Progress

Component	What exists	Status
Menu and data	• About 70 menu items, most with photos • Calories are real, prices are estimates	Done, prices to replace
Web terminal and agent	• Presence trigger, speech in and out • Agent searches the menu, edits the cart, and confirms orders • Runs on rules, a local model, or a hosted model • Automated tests pass, including a price check	In place, being tested
ML service	• Endpoints for signs, recommendations, and health • Splits signing into separate signs, with unit tests	Done
ASL datasets	• Two public datasets reviewed (ASL Citizen and Google ISLR) • Most of our ordering words are covered • Only four signs need our own recordings • Landmark data is small enough for one GPU	Done
ASL model (first version)	• Works on 64 signs, about 89% top-1 and 95% top-5 on new signers • Accuracy varies a lot between signers • Exported to ONNX and runs in a few milliseconds	In progress

Component	What exists	Status
Order recommender	• Simple baseline coded • Training planned for Sprints 3 to 5	Baseline done

8.3 Development Tools

Member	Tools	Used for
All members	• GitHub, Node, Python, Chrome, and the run guide	Everything
Raymond Li	• PyTorch, MediaPipe, ONNX Runtime, Kaggle API • Colab or Kaggle GPU • YOLO and ByteTrack	ASL model (M2), traffic detection (T1), shared model (S1)
Jeremy Tung	• Next.js, TypeScript, Tailwind • Vercel AI SDK, PGlite, Web Speech API	Terminal and agent (M0, M1), recording tool (M3), dashboard (T5)
Jack Liang	• Ollama • Hugging Face with PEFT for QLoRA • llama.cpp for GGUF export • PyTorch	Recommender (M4), fine-tuned LLM (M5), forecast (T2), signal agent (T3)
Sophia Atendido	• Chrome DevTools • SUMO • axe and VoiceOver screen reader	Speed testing (R1), simulator (T4), accessibility (M6)

8.4 Known Issues & Next Steps

Known issues

- Getting landmarks from the videos is slow (more than a day for the full dataset).
- The model's confidence score is not reliable yet.
- Accuracy varies a lot between signers, which we will report as a fairness finding.
- Traffic results may not carry over to other kinds of intersections.

Next steps

- Re-test with signers the model never saw.
- Measure speed on a live webcam.
- Train the recommender and replace the estimated prices.
- Pick traffic data we are allowed to use and set up the simulator.

8.5 Prototype Plan

We want to try every tool on a small example early, so the use of each technology is understood before we need it. Each test has an owner and a target date.

Tool	First small test	Owner	Target
ASL model on the webcam	Run the ONNX model through /ws/sign (in progress)	Raymond	Oct 11
Free GPU (Colab or Kaggle)	Train a short ASL run on it	Raymond	Oct 11
Local LLM (Ollama)	Run npm run benchmark:agent with the local model	Jack	Oct 25
QLoRA	Make a few dialogues with ml/llm/synthesize.py and fine-tune on Colab for a few steps	Jack	Oct 25
SUMO	Install it and run a sample intersection	Sophia	Oct 25
Accessibility (axe)	Run axe on the terminal page and see what it finds	Sophia	Nov 8
YOLO and ByteTrack	Count cars in one short sample video	Raymond	Nov 22
Shared model	Add a second input layer and output head to the ASL model and run it on sample data	Raymond	Dec 6
LLM safety check	Write a check that rejects unsafe signal timings, like our price check	Jack	Dec 6

Chapter 9: Project Schedule (Sophia Atendido)

We work in two-week sprints. Each sprint ends with a team review and a short demo, and every second sprint ends with an advisor check-in.

9.1 Sprint Calendar & Checkpoints

Sprint	Raymond	Jeremy	Jack	Sophia
Sprint 1 Sep 14 to Sep 27	ASL datasets and landmark extraction (M2)	Walking skeleton, menu, and agent (M0, M1)	Local model in Ollama as an agent backend (M1)	Presence trigger and spoken replies (M0)
Sprint 2 Sep 28 to Oct 11	ASL model running live, short GPU training run (M2)	Agent tests, touch and speech ordering (M1)	Recommender baseline and data (M4)	Measure sign speed on a live webcam (R1)
Sprint 3 Oct 12 to Oct 25	Train on more signers, re-test on new signers (M2)	Real menu prices (M1)	Start recommender training, local LLM benchmark (M4)	Better sign detection, SUMO sample run (R1)
Sprint 4 Oct 26 to Nov 8	Improve the ASL model (M2)	Recording tool (M3)	Recommender, QLoRA test run (M4)	Make the confidence score reliable, first axe check (R1)
Sprint 5 Nov 9 to Nov 22	Record custom signs, count cars in a sample video (M3, T1)	Record custom signs, organize recordings (M3)	Record custom signs, finish recommender (M3, M4)	Record custom signs, fix axe findings (M3, R1)
Sprint 6 Nov 23 to Dec 6	Retrain with new signs, shared model test (M2)	Put it together, rehearse, 295A demo	Recommender in the terminal, timing safety check draft (M4)	Speed check on the demo build, demo script (R1)
Winter break Dec 7 to Jan 24	Traffic data and detector setup (T1)	Dashboard layout and data format (T5)	Practice dialogues (M5)	Simulator setup (T4)
Sprints 7 and 8 Jan 25 to Feb 21	Traffic counts (T1)	Dashboard starts (T5)	LLM fine-tuning and comparison (M5)	Sample intersection in the simulator (T4)
Sprints 9 and 10 Feb 22 to Mar 21	Start the shared model (S1)	Dashboard (T5)	Traffic forecast (T2)	Fixed-timing baseline (T4)
Sprints 11 and 12	Shared model testing (S1)	Dashboard finished (T5)	Signal agent with safety check (T3)	Accessibility audit and speed tuning (M6)

Sprint	Raymond	Jeremy	Jack	Sophia
Mar 22 to Apr 18				
Sprints 13 and 14 Apr 19 to May 16	Shared model results for the report	Final integration	Signal agent results for the report	Simulation study

All members work on the report, workbooks, and final demo in Sprints 13 and 14.

9.2 Milestones

Milestone	Lead	Approx. period	Goal	Status
M0 Walking skeleton	Jeremy	Sep 2026	Camera input to server to agent to spoken reply, with timing logs.	Mostly in place
M1 Menu and agent	Jeremy	Sep-Oct 2026	Menu, ordering agent, and touch and speech ordering, with tests.	In progress
M2 ASL model	Raymond	Sep-Dec 2026	Train the ASL model, connect it to the live camera feed, and retrain with the custom signs.	In progress
R1 Real-time performance	Sophia	Oct-Dec 2026	Check how fast it is, make sign detection and the confidence score more reliable.	Not started
M4 Recommender	Jack	Oct-Dec 2026	Train the recommender, compare it to the baseline, and add it to the terminal.	Not started
M3 Custom signs	Jeremy	Nov 2026	Build the recording tool and have everyone record the missing signs.	Not started
295A demo	Jeremy	Dec 2026	Order in ASL start to finish on a laptop webcam.	Planned
T1 Traffic data and detection	Raymond	Dec 2026-Feb 2027	Pick traffic data, then detect and track vehicles to get counts.	Not started
M5 Fine-tuned LLM	Jack	Dec 2026-Feb 2027	Make practice dialogues, fine-tune the model, and compare it to the hosted model.	Not started
T4 Simulator and baseline	Sophia	Dec 2026-Mar 2027	Set up the simulator and a fixed-timing baseline.	Not started
T5 Traffic dashboard	Jeremy	Feb-Apr 2027	A dashboard that shows the video, counts, forecast, and signal plan.	Not started

Milestone	Lead	Approx. period	Goal	Status
T2 Traffic forecast	Jack	Mar 2027	Prepare traffic data and test the forecast.	Not started
S1 Shared model	Raymond	Mar-Apr 2027	Train one model on both tasks and check it on data it has not seen.	Not started
T3 LLM signal agent	Jack	Apr 2027	LLM suggests signal timing and code checks that it is safe.	Not started
M6 Accessibility and polish	Sophia	Apr 2027	Accessibility checks, speed tuning, and demo script.	Not started
Final integration and report	All	May 2027	Simulation study, final integration, report, and final demo.	Planned

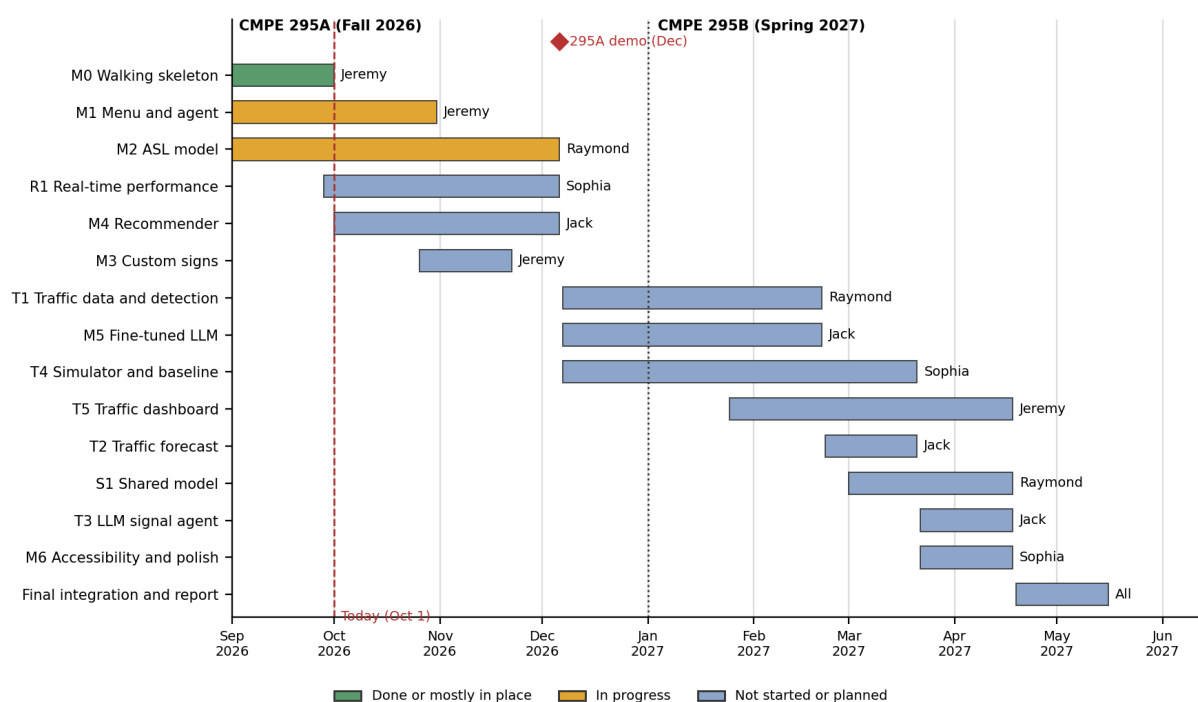


Figure 9.1: Project Gantt chart. Bars show each milestone, the labels show who leads it, and the dashed line is today.

9.3 Tasks

Member	Task	Milestone
Raymond Li (ML and datasets)	Datasets, landmarks, and ASL model training	M2
	Traffic video and vehicle detection	T1

Member	Task	Milestone
	Shared model training and testing	S1
Jeremy Tung (team lead, app pipeline)	Web terminal, agent, and menu	M0, M1
	Recording tool for custom signs	M3
	Putting it together and the demo	295A demo
	Traffic dashboard	T5
	Final integration	Final
Jack Liang (recommender and LLM)	Recommender	M4
	Dialogues and LLM fine-tuning	M5
	Traffic forecast model	T2
	LLM signal agent with safety checks	T3
Sophia Atendido (real-time and accessibility)	Speed testing, sign detection tuning, and confidence score	R1
	Traffic simulator and baseline	T4
	Accessibility audit and presence trigger	M6
	Simulation study	Final
All	Record the custom signs, workbooks, report, and presentations	Ongoing